Nano Matter User Manual

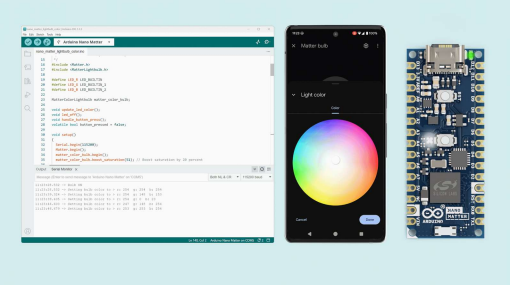
Learn about the hardware and software features of the Arduino® Nano Matter.

Author • Christopher Mendez

Last revision • 10/15/2025

Overview

This user manual will guide you through a practical journey covering the most interesting features of the Arduino Nano Matter. With this user manual, you will learn how to set up, configure and use this Arduino board.



Hardware and Software Requirements

Hardware Requirements

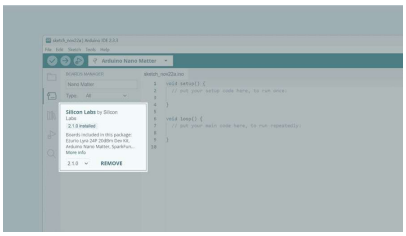
- Nano Matter (x1)
- USB-C® cable (x1)

Software Requirements

- Arduino IDE 2.0+ or Arduino Cloud Editor

Board Core and Libraries

The **Silicon Labs** core contains the libraries and examples you need to work with the board's components, such as its Matter, Bluetooth® Low Energy, and I/Os. To install the Nano Matter core, navigate to **Tools > Board > Boards Manager** or click the Boards Manager icon in the left tab of the IDE. In the Boards Manager tab, search for `Nano Matter` and install the latest `Silicon Labs` core version.



Installing the Silicon Labs core in the Arduino IDE

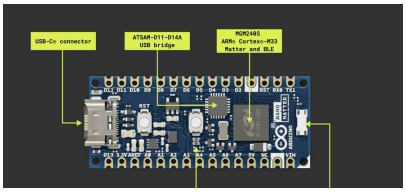
Product Overview

The Nano Matter merges the well-known Arduino way of making complex technology more accessible with the powerful MGM240S from Silicon Labs, to bring Matter closer to the maker world, in one of the smallest form factors in the market.

It enables 802.15.4 (Thread®) and Bluetooth® Low Energy connectivity, to interact with Matter-compatible devices with a user-friendly software layer ready for quick prototyping.

Board Architecture Overview

The Nano Matter features a compact and efficient architecture powered by the MGM240S (32-bit Arm® Cortex®-M33) from Silicon Labs, a high-performance wireless module optimized for the needs of battery and line-powered IoT devices for 2.4 GHz mesh networks.



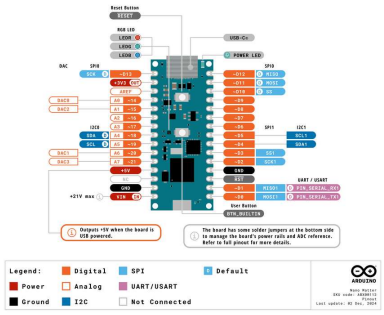
Nano Matter's main components

Here is an overview of the board's main components, as shown in the image above:

Microcontroller: at the heart of the Nano Matter is the MGM240S, a high-performance wireless module from Silicon Labs. The MGM240S is built around a 32-bit Arm® Cortex®-M33 processor running at 78 MHz.

Wireless connectivity: the Nano Matter microcontroller also features multiprotocol connectivity to enable Matter IoT protocol and Bluetooth® Low Energy. This allows the Nano Matter to be integrated with smart home systems and communicate wirelessly with other devices.

Pinout



Nano Matter Simple pinout

The full pinout is available and downloadable as PDF from the link below:

Nano Matter pinout

Datasheet

The complete datasheet is available and downloadable as PDF from the link below:

Nano Matter datasheet

Schematics

The complete schematics are available and downloadable as PDF from the link below:

STEP Files

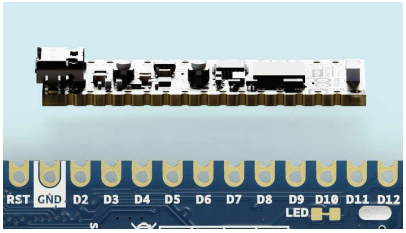
The complete STEP files are available and downloadable from the link below:

- Nano Matter STEP files

Form Factor

The Nano Matter (ABX00112) board features castellated pins, which are ideal for integrating the board into final solutions.

You can easily solder the Nano Matter (ABX00112) in your custom PCB, since the board does not present any bottom-mounted components.



Nano Matter (ABX00112) castellated pins

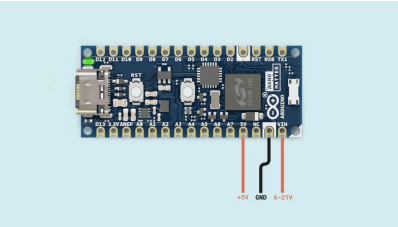
The Nano Matter with headers (ABX00137) out of the box is also available, providing easy access for probing and testing through the headers.

First Use

Powering the Board

The Nano Matter can be powered by:

- A USB-C® cable (not included).
- An external **5 V power supply** connected to `5V` pin (please, refer to the board pinout section of the user manual).
- An external **6-21 V power supply** connected to `VIN` pin (please, refer to the board pinout section of the user manual).



Nano Matter externally powered

For low-power consumption applications, the following hacks are recommended:

- Cut the power status LED jumper off to save energy.
- Power the board with an external **3.3 V power supply** connected to **3.3V** pin. This will not power the *USB bridge IC*, so more energy will be saved.

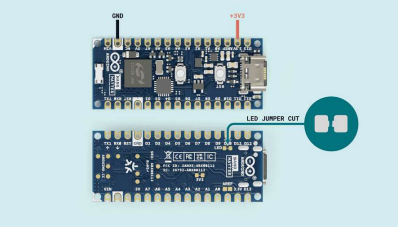
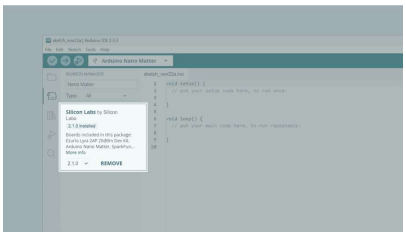


Image showing the LED jumper and external 3.3 V power

To power the board through the VIN pin you need to close the jumper pads with solder. The maximum voltage supported is +5 VDC.

Install Board Core and Libraries

The **Silicon Labs** core contains the libraries and examples you need to work with the board's components, such as its Matter, Bluetooth® Low Energy, and I/Os. To install the Nano Matter core, navigate to **Tools > Board > Boards Manager** or click the Boards Manager icon in the left tab of the IDE. In the Boards Manager tab, search for `Nano Matter` and install the latest `Silicon Labs` core version.



Installing the Silicon Labs core in the Arduino IDE

Hello World Example

Let's program the Nano Matter with the classic `hello world` example typical of the Arduino ecosystem: the `Blink` sketch. We will use this example to verify that the board is correctly connected to the Arduino IDE and that the Silicon Labs core and the board itself are working as expected.

Copy and paste the code below into a new sketch in the Arduino IDE.

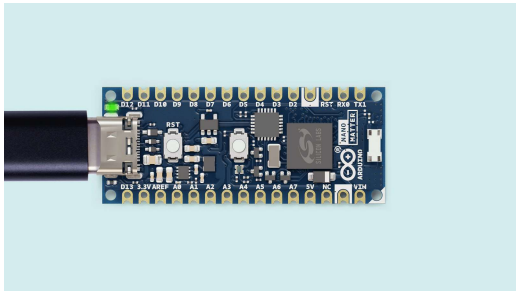
```
// the setup function runs once w
void setup() {
  // initialize digital pin LED_B
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over an
void loop() {
  digitalWrite(LED_BUILTIN, HIGH)
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}
```

In the Nano Matter, the `LED_BUILTIN` macro represents the **red LED** of the built-in RGB LED of the board. Please refer to the image below.

To upload the code to the Nano Matter, click the **Verify** button to compile the sketch and check for errors; then click the **Upload** button to program the board with the sketch.

Uploading a sketch to the Nano Matter in the Arduino IDE

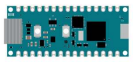
You should now see the red LED of the built-in RGB LED turning on for one second, then off for one second, repeatedly.



If everything works as expected, you are ready to continue searching and experimenting with this mighty board.

Matter

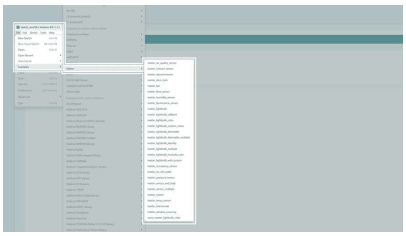
Developing Matter-compatible IoT solutions has never been easier with the Arduino ecosystem.



Nano Matter

The Nano Matter can communicate with Matter hubs through a Thread® network, so the hubs used must be **Thread® border routers**.

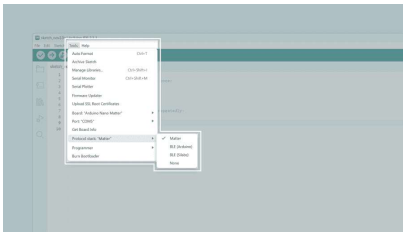
The Silicon Labs core in the Arduino IDE comes with several Matter examples ready to be tested with the Nano Matter and works as a starting point for almost any IoT device we can imagine building.



Matter examples

The *matter_lightbulb* example is the only officially Matter-certified profile for the Nano Matter. Consequently, while running any of the other available profile examples, it is expected to get an *Uncertified device* message in the different Matter-compatible apps. This does not prevent the user from prototyping a solution with different configurations.

First, to start creating *Matter-enabled* solutions, we need to select the Matter protocol in **Tools > Protocol stack > Matter**:



Matter Protocol stack selected

In the example below, we are going to use the Nano Matter as a *RGB Lightbulb*. For this, navigate to **File > Examples > Matter** and open the built-in sketch called **nano_matter_lightbulb_color**.

```
#include <Matter.h>
#include <MatterLightbulb.h>

#define LED_R LED_BUILTIN
#define LED_G LED_BUILTIN_1
#define LED_B LED_BUILTIN_2

MatterColorLightbulb matter_col

void update_led_color();
void led_off();
void handle_button_press();
volatile bool button_pressed =

void setup()
{
  Serial.begin(115200);
  Matter.begin();
  matter_color_bulb.begin();
  matter_color_bulb.boost_satur

  // Set up the onboard button
  pinMode(BTN_BUILTIN, INPUT_PU
  attachInterrupt(BTN_BUILTIN,

  // Turn the LED off
  led_off();
```

Here is the example sketch main functions explanation:

In the `setup()` function, Matter is initialized with `Matter.begin()` alongside the initial configurations of the board to handle the different inputs and outputs.

The device commissioning is verified with `Matter.isDeviceCommissioned()` to show the user the network pairing credentials if needed, and the connection is confirmed with the `Matter.isDeviceThreadConnected()` function.

With the `matter_color_bulb.is_online()` function, we confirm that the device is online and reachable by the coordinator app.

In the `loop()` function, the RGB LED is controlled on and off with

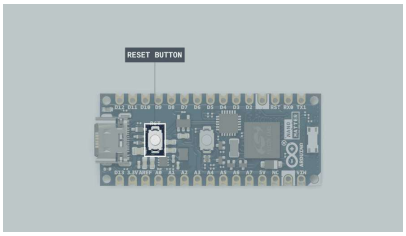
`matter_color_bulb.get_onoff()` and the button state is read to control the LED manually.

In the `update_led_color()` function, the color defined in the app is retrieved using the function `matter_color_bulb.get_rgb(&r, &g, &b)` that stores the requested color code in RGB format variables.

To upload the code to the Nano Matter, click the **Verify** button to compile the sketch and check for errors; then click the **Upload** button to program the board with the sketch.

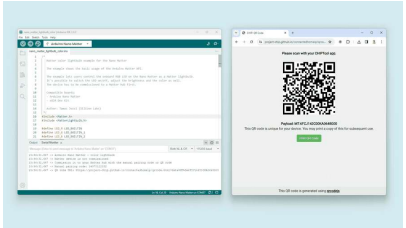
Upload the Matter RGB example

After the code is uploaded, open the Arduino IDE Serial Monitor and reset the board by clicking on the reset button. To commission a Matter device to the network you will need the credentials shown in the terminal.



Nano Matter Reset Button

You will find a **Manual pairing code** and a **QR code URL** as follows:



Open the QR code URL on your browser to generate the QR code.

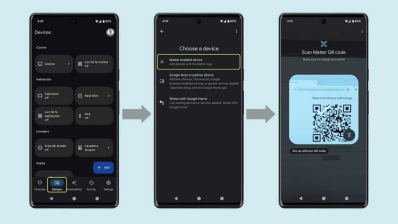
With Google Home™

To create your first IoT device with the Nano Matter and the Google Home ecosystem, you first need to have a Matter-compatible hub. The Google Home products that can work as a **Matter hub** through **Thread®** are listed below:

- Nest Hub (2nd Gen)
- Nest Hub Max
- Nest Wifi Pro (Wi-Fi 6E)
- Nest Wifi

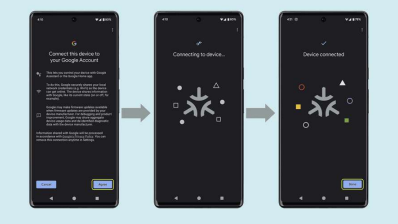
Other Google devices are compatible with Matter but not Thread®.

To commission your device, open the Google Home app, navigate to devices, click on **add device** and select the **matter-enabled device** option:



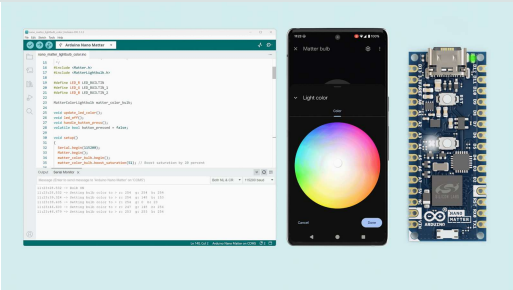
Adding a new Matter device to Google Home

Then, wait for the device to be commissioned and added to the Google Home app:



Device added

Finally, you will be able to control the Nano Matter built-in RGB LED as a native smart



You are also able to control your device using voice commands with your personal assistant.

If you want to commission your Nano Matter solution with another service, follow the steps in the decommissioning section.

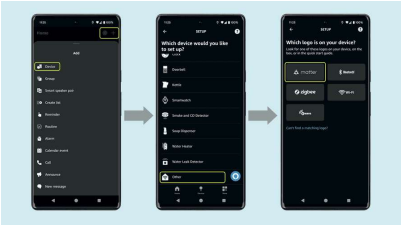
With Amazon Alexa

The Amazon Alexa products that can work as a **Matter hub** through **Thread®** are listed below:

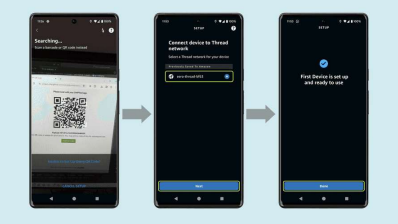
- Echo (4th Gen)
- Echo Show 10 (3rd Gen)
- Echo Show 8 (3rd Gen)
- Echo Hub
- Echo Studio (2nd Gen)
- Echo Plus (2nd Gen)
- eero Pro 6 and 6E
- eero 6 and 6+
- eero PoE 6 and gateway
- eero Pro
- eero Beacon
- eero Max 7

Other Amazon devices are compatible with Matter but not Thread®.

To commission your device, open the Amazon Alexa app, click on the upper right **+** symbol, select **device** and select the **Matter** logo option:

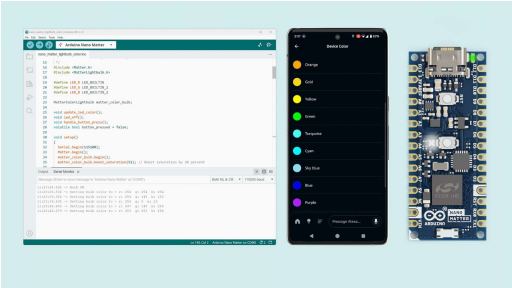


Read the QR code generated by the Nano Matter sketch, select the **Thread®** network available and then wait for the device to be commissioned and added to the Alexa app:



Device added

Finally, you will be able to control the Nano Matter built-in RGB LED as a native smart device. You can turn it on and off and control its color and brightness.



You are also able to control your device using voice commands with your personal assistant.

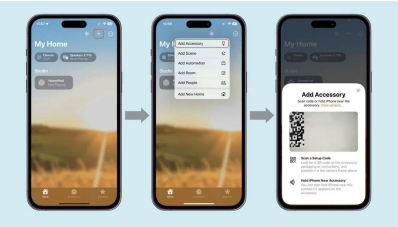
If you want to commission your Nano Matter solution with another service, follow the steps in the decommissioning section.

With Apple Home

The Apple Home products that can work as a **Matter hub** through **Thread®** are listed below:

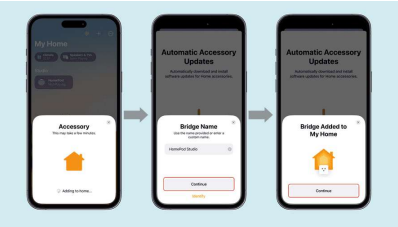
- Apple TV 4K (3rd generation) Wi-Fi + Ethernet
- Apple TV 4K (2nd generation)
- HomePod (2nd generation)
- HomePod mini

To commission your device, open the Apple Home app, click on the upper right **+** symbol, select **Add Accessory** and scan the **QR code** generated by the Nano Matter in the Serial Monitor:



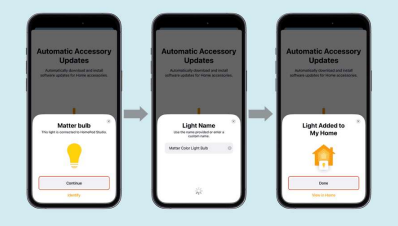
Adding a new Matter device to Apple Home

If this is your first Matter device, you may be asked to define the Matter hub to be used. Select its house location and give it a name:



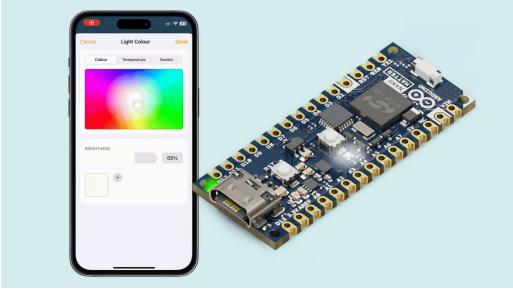
Selecting the Matter hub (Bridge)

Then, follow the steps for adding and naming the Matter Light bulb:



Adding the Light bulb

Finally, you will be able to control the Nano Matter built-in RGB LED as a native smart device. You can turn it on and off and control its color and brightness.



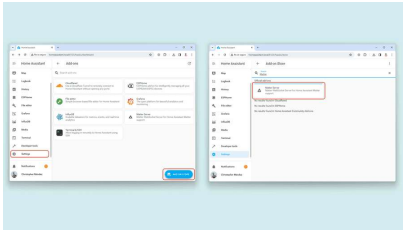
You are also able to control your device using voice commands with your personal assistant

If you want to commission your Nano Matter solution with another service, follow the steps in the decommissioning section.

With Home Assistant

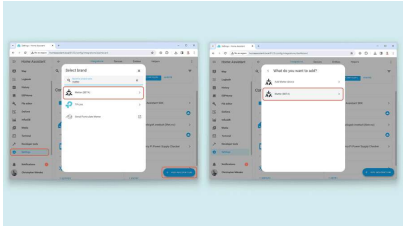
To use Matter with Home Assistant, you will need one of the *Google Home* or *Apple Home* devices that can work as a **Thread® Border Router**, as listed in the previous sections.

To set up Home Assistant so that it can manage Matter devices, we need first to install the **Matter Server** add-on. For this, navigate to **Settings > Add-Ons > Add-On Store** and search for **Matter server**:



Installing the Matter Server

When the Matter server is correctly installed, navigate to **Settings > Devices & Services > Add Integration** and search for **Matter**:



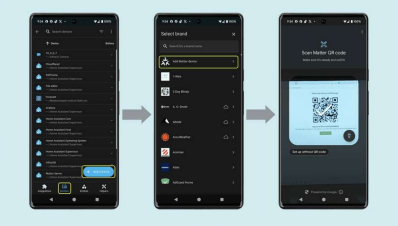
Installing the Matter integration

A prompt will show up asking for a connection method; if you are working with custom containers running the Matter server, uncheck the box.

In our case, we leave it checked as the Matter server is running in Home Assistant.

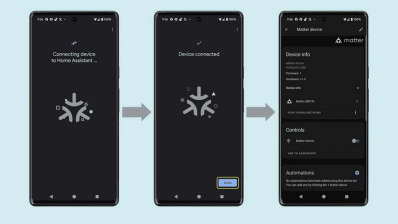
You will receive a **Success** message if everything is okay. With the Home Assistant integration ready, we can start with the Nano Matter commissioning

To commission your device, open the Home Assistant app on your smartphone, go to **Settings > Devices & Services**, in the lower tabs bar, go to **Devices** and tap on **Add Device**. Tap on **Add Matter device** and scan the QR code generated by the Nano Matter in the Serial Monitor:



Adding a new Matter device to Home Assistant

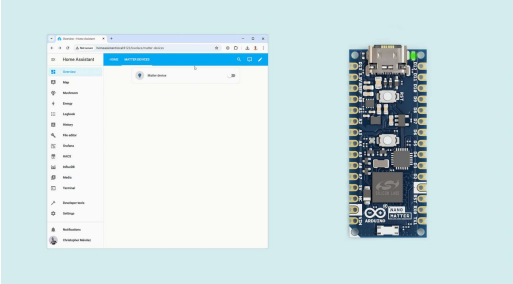
Then, wait for the device to be commissioned and added to the Home Assistant app:



Device Added

Finally, you will be able to control the Nano Matter built-in RGB LED as a native smart device. You can turn it on and off and control its color and brightness.

You can use the Home Assistant web view or mobile app to control your device.



If you want to commission your Nano Matter solution with another service, follow the steps in the **decommissioning** section.

We tested the Matter integration with the Home Assistant OS and hosting Home Assistant in Docker containers.

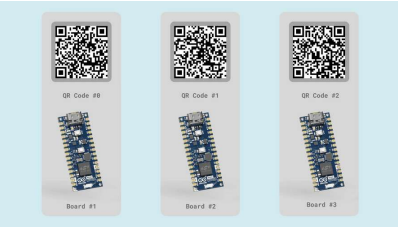
Home Assistant Tips

- Make sure you are using a **64-bit** Home Assistant version (OS or Docker containerized version).
- Use the **Thread®** add-on to verify your available Thread® networks.
- You can just have a Matter device commissioned to one platform at a time.

Be aware that the Matter integration for Home Assistant is still in BETA, it can receive major updates and its functionality may vary between different vendors.

Updating the Commissioning QR Code

Each Nano Matter board comes with a default QR code used for commissioning. In this section, you will learn how to **generate a unique QR code** for your device by updating its provisioning ID.



Unique QR Codes for your Matter devices

By assigning a unique provisioning ID, you can:

- Generate a distinct QR code for each board.
- Commission multiple Nano Matter boards to the same network without conflicts.
- Prepare your devices for real-world field deployment.

Prerequisites

Before starting, make sure you have the following:

- Make sure the **Arduino IDE** and the **Silicon Labs Arduino Core** are both installed.
- Make sure there is only **one** board connected to your computer at a time.
- Your Matter sketch already flashed to the board.
- Clone the Arduino Matter Provision Tool repository on your local machine.

Changing the Provisioning ID

To assign a new provisioning ID and generate a new QR code:

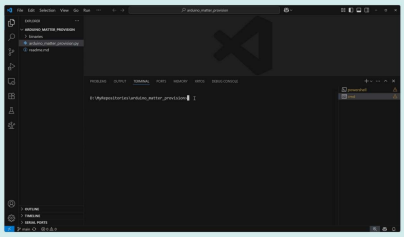
- Open a terminal and navigate to the cloned `/arduino_matter_provision` folder.
- The provisioning command has the following format:

```
1  python arduino_matter_provisi
```

- Replace `<board_name>` with `nano_matter` and choose a configuration number (e.g., `1`):

```
1  python arduino_matter_provisi
```

- Run the script to change the provisioning data using the given structure.



Once the script finishes, open the **Arduino Serial Monitor**. You will see the updated commissioning credentials there, no need to re-upload the sketch.

Here's what the new credentials might look like:

Manual Pairing Code: `00417637863`

QR code URL:

```
https://project-chip.github.io/connectedhomeip/qrcode.html?data=MT%3A8YT00-D000CQ-01VB10
```

Make sure all your Nano Matter boards has been configured with a different ID.

Device Decommissioning

If you have a Matter device configured and working with a *specific platform*, for example with the Google Home ecosystem, and you want to integrate it with Alexa or Apple Home instead, you need to decommission it first from the previous service.

In simple terms, **decommissioning** refers to unpairing your device from a current service to be able to pair it with a new one.

Use the library built-in example found in **File > Examples > Matter > matter_decommission**.

You can integrate this method in your solutions to leverage the Nano Matter built-in button to handle decommissioning:

```
#include <Matter.h>

void setup() {
  // put your setup code here, t
  Serial.begin(115200);

  Matter.begin();
  pinMode(BTN_BUILTIN, INPUT_PUL
  pinMode(LEDR, OUTPUT);
  digitalWrite(LEDR, HIGH);
}

void loop() {
  // put your main code here, to
  decommission_handler();
}


void decommission_handler() {
  if (digitalRead(BTN_BUILTIN) =
    // measures time pressed
    int startTime = millis();
    while (digitalRead(BTN_BUILT

      int elapsedTime = (millis(

      if (elapsedTime > 10) {
        Serial.printf("Decommiss
```

The sketch above allows you to decommission your board manually after **pressing** the Nano Matter user button for **10 seconds**. You can monitor the status in the Arduino IDE Serial Monitor.

Arduino Cloud

The Nano Matter has no built-in Wi-Fi® but can be seamlessly integrated with the Arduino Cloud using its API and Matter.

We are going to use the Home Assistant Matter integration to create automations and scripts that help us forward the Nano Matter data to the Arduino Cloud.

In case it is the first time you are using the Arduino Cloud:

> To use the Arduino Cloud, you need an account. If you do not have an account, create one for free here.
>
> See the Arduino Cloud plans and choose

As a practical example, we are going to use the Nano Matter CPU temperature sensor and send the data to Arduino Cloud for monitoring. We will leverage the variety of widgets to create a professional and nice-looking user interface.

Nano Matter Programming

The application sketch below is based on the `matter_temp_sensor` example that can be also found in **File > Examples > Matter**. This variation includes the decommission feature to show it implemented in a real application.

```
#include <Matter.h>
#include <MatterTemperature.h>

MatterTemperature matter_temp_se

void setup()
{
  Serial.begin(115200);
  Matter.begin();

  pinMode(BTN_BUILTIN, INPUT_PUL
  pinMode(LEDR, OUTPUT);
  digitalWrite(LEDR, HIGH);

  matter_temp_sensor.begin();

  Serial.println("Matter tempera

  if (!Matter.isDeviceCommissior
    Serial.println("Matter devic
    Serial.println("Commission i
    Serial.printf("Manual pairir
    Serial.printf("QR code URL:
  }
  while (!Matter.isDeviceCommiss
    delay(200);
    decommission_handler();  //
  }
```

The main code functions are explained below:

The temperature sensor object is created with the `MatterTemperature matter_temp_sensor;` statement. To initiate it, in the `setup()` function, we used `matter_temp_sensor.begin();`

The `decommission_handler()` lets us

The microcontroller's internal temperature is measured with the function `getCPUTemp();`.

The temperature value is advertised using the `matter_temp_sensor.set_measured_value_celsius(current_cpu_temp);` function.

After uploading the code to the Nano Matter, verify it is decommissioned from any other service previously used. For this, open the Serial Monitor and reset the board by clicking on the reset button.

If it is not decommissioned you will see temperature readings printed in the Serial Monitor. To decommission it, follow these steps:

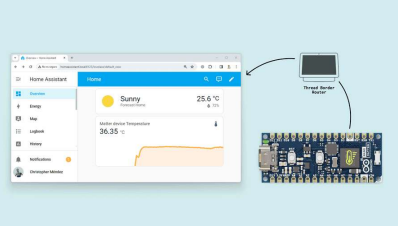
Press the user button for **10 seconds** until the board's built-in LED starts **blinking in red**. You will also see a message confirming the process in the Serial Monitor.

Finally, reset the board by clicking on the reset button and you should see the Matter commissioning credentials in the Serial Monitor.

Device Commissioning

Now it is time to commission the Nano Matter with Home Assistant, for this, follow the steps explained in this **section**.

Once you have everything set up and running you will be able to monitor the Nano Matter temperature in Home Assistant:



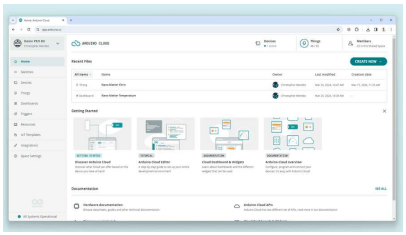
Nano Matter Temperature in Home Assistant

Be aware that the Matter integration for Home Assistant is still in BETA, it can receive major updates and its functionality may vary between different vendors.

Arduino Cloud Set-Up

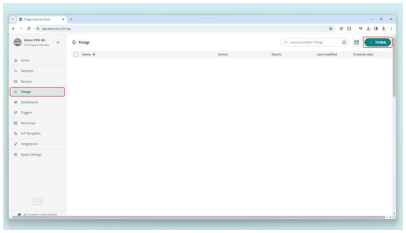
Let's walk through a step-by-step demonstration of how to set up the Arduino Cloud.

Log in to your Arduino Cloud account; you should see the following:



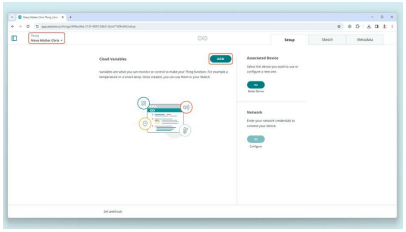
Arduino Cloud Initial Page

Navigate to **Things** in the left bar menu and click on **+ Thing** to add a Thing:



Creating a Thing

Give your Thing a name and click on **ADD** to add the temperature variable:

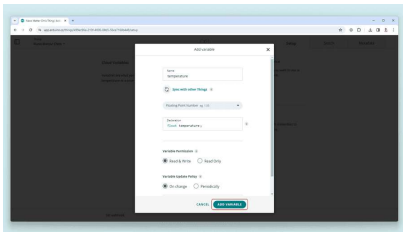


Adding a variable

Define the variable with the following settings:

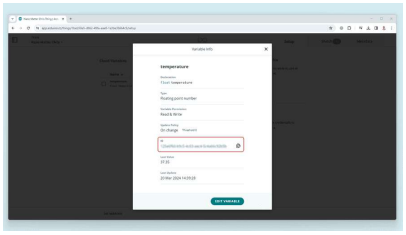
Name: temperature

Update Policy: On change



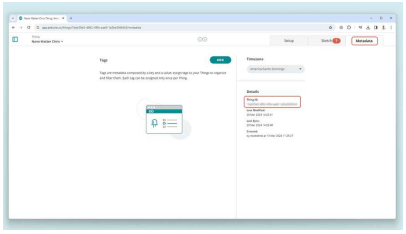
Setting up the variable

Click on the created variable and copy its **ID**, we will need it later.



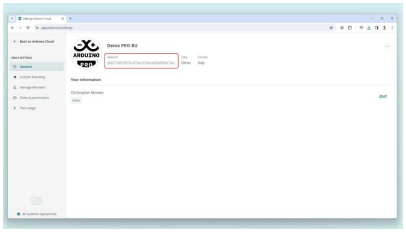
Variable ID

Navigate to your Thing metadata and copy the **Thing ID**, we will need it later.



Thing ID

In the left bar menu, navigate to **Space Settings** and copy the **Space ID**, we will need it later.



Space ID

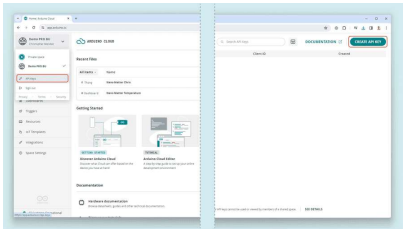
If you can't see the *Space Settings* section is because you are using an Arduino Cloud free plan, check the **plans** with the API feature enabled.

At this point you should have three IDs related to your project:

- Variable ID
- Thing ID
- Space ID

To properly authenticate the requests we are going to use to upload the data to the Arduino Cloud we need to create **API Keys**.

For this, navigate to **API Keys** in the upper left corner drop-down menu and click on **Create API Key**:



API Keys generation

You should get a **Client ID** and a **Client Secret**. Save these credentials in a safe place, you will not be able to see them again.

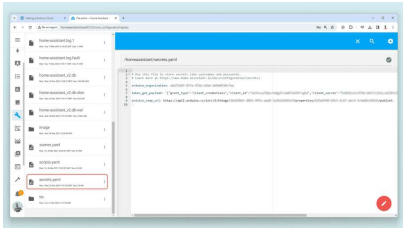
Home Assistant Set-Up

Now, let's configure Home Assistant to set the forwarding method to Arduino Cloud.

First, we are going to save and define our project IDs, credentials and Keys in a safe place inside the Home Assistant directory called **secrets.yaml**. Use the *File Editor* Add-on to easily edit this file, and format the data as follows:

```
1 arduino_organization: <Space ID>
2
3 token_get_payload: '{"grant_type":
4
```

This will let us use this data without exposing it later.



Secrets.yaml file to store credentials

Now, let's define the services that will help us do the HTTP requests to Arduino Cloud sending the temperature value to it. Using the **File Editor** navigate to the **configuration.yaml** file and add the following blocks:

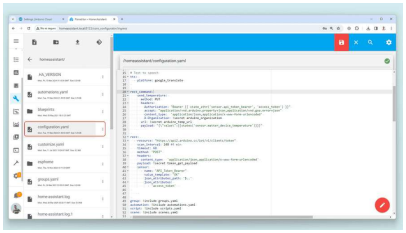
```
rest:
  - resource: "https://api2.arduin
    scan_interval: 240 #4 min
    timeout: 60
    method: "POST"
    headers:
      content_type:  'application
    payload: !secret token_get_pa
    sensor:
      - name: "API_Token_Bearer"
        value_template: "OK"
        json_attributes_path: '$.
        json_attributes:
          - 'access_token'
```

The RESTful integration lets us periodically gather from our Arduino Cloud account a **token** that is mandatory to authenticate our requests. This token expires every 5 minutes, this is why we generate it every 4 minutes. The token is stored in a sensor attribute called **API_Token_Bearer**.

```
rest_command:
    send_temperature:
      method: PUT
      headers:
        Authorization: "Bearer {{
        accept: "application/vnd.
        content_type:  'applicati
        X-Organization: !secret a
      url: !secret arduino_temp_u
      payload: "{\"value\":{{stat
```

The RESTful command integration lets us define the HTTP request structure to be able to send the Nano Matter temperature sensor data to the Arduino Cloud. We can call this service from an automation.

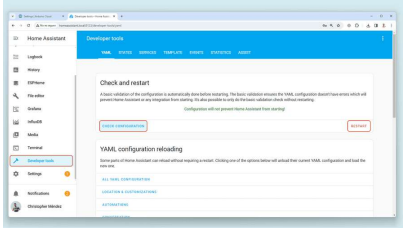
As you may noticed, the sensitive data we stored in the "secrets.yaml" file is being called here with the `!secret` prefix.



configuration.yaml file to define services

To learn more about the Arduino Cloud API, follow this guide.

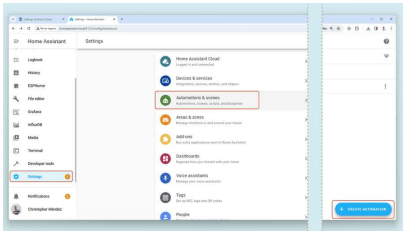
For the changes to take effect, navigate to **Developers Tools** and click on **Check Configuration**, if there are no errors, click on **Restart** and restart Home Assistant.



Restarting Home Assistant

Finally, let's set up the automation that will call the **send_temperature** service every time the

For this, navigate to **Settings > Automations & scenes** and click on **Create Automation**.

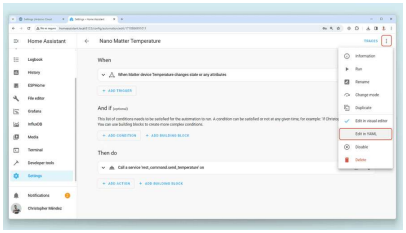


Automation setting

In the upper right corner, click on the "three dots" menu and select **Edit in YAML**, replace the text there with the following:

```
alias: Nano Matter Temperature
description: ""
trigger:
  - platform: state
    entity_id:
      - sensor.matter_device_temp
condition: []
action:
  - service: rest_command.send_te
    data: {}
mode: single
```

This automation will be triggered if the Nano Matter sensor temperature value changes and will act by calling the **rest_command.send_temperature** service.

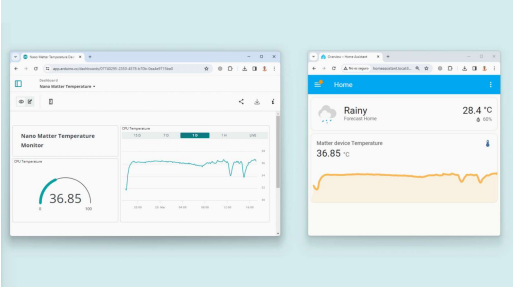


Automation defining

If you use different names for the services or devices, make sure to update them in the YAML code above.

Final Results

With this done, Home Assistant should be forwarding the Nano Matter sensor data to Arduino Cloud where you can monitor it with different widgets and charts as follows:

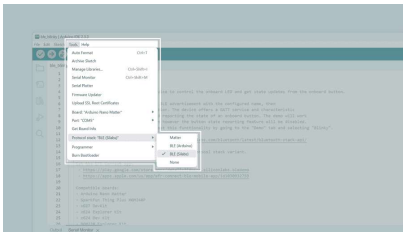


Bluetooth® Low Energy

Silicon Labs BLE Library

To enable Bluetooth® Low Energy communication on the Nano Matter, you must enable the "BLE" protocol stack in the Arduino IDE board configurations. In this section we will use the official Silabs Bluetooth library.

In the upper menu, navigate to **Tools > Protocol stack** and select **BLE(Silabs)**.



Enable "BLE(Silabs)" Protocol Stack

For this Bluetooth® Low Energy application example, we are going to control the Nano Matter built-in LED and read the onboard button status. The example sketch to be used can be found in **File > Examples > Silicon Labs > ble_blinky**:

```
/*
   BLE blinky example
 */

bool btn_notification_enabled =
volatile bool btn_state_changed
volatile uint8_t btn_state = LO
static void btn_state_change_ca
static void send_button_state_r
static void set_led_on(bool sta

void setup()
{
  pinMode(LED_BUILTIN, OUTPUT);
  digitalWrite(LED_BUILTIN, LED
  set_led_on(false);
  Serial.begin(115200);
  Serial.println("Silicon Labs

  // If the board has a built-i
  #ifdef BTN_BUILTIN
  pinMode(BTN_BUILTIN, INPUT_PU
  attachInterrupt(BTN_BUILTIN,
  #else // BTN_BUILTIN
  // Avoid warning for unused f
  (void)btn_state_change_callba
  #endif // BTN_BUILTIN
}
```

Here are the main functions explanations of the example sketch:

The `sl_bt_on_event(sl_bt_msg_t *evt)` function is the main one of the sketch and it is responsible for:

- Initiating the Bluetooth® Low Energy radio, and start advertising the defined services alongside its characteristics.
- Handling the opening and closing of connections.
- Handling incoming and outgoing Bluetooth® Low Energy messages.

The `ble_initialize_gatt_db()` function is responsible for:

- Adding the Generic Access service with the `1800` UUID.
- Adding the Blinky service with the `de8a5aac-a99b-c315-0c80-60d4cbb51224` UUID.
- Creating the device name characteristic so we can find the device as "Blinky Example" with the `2A00` UUID.

`5b026510-4088-c297-46d8-be6c736a087a` UUID.

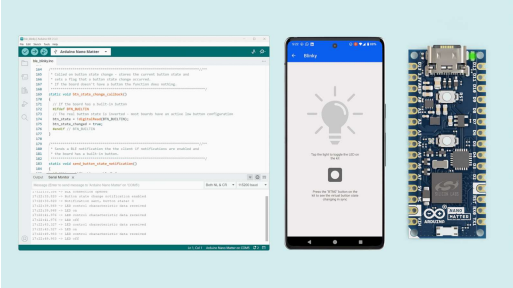
Adding the Button report characteristic to the Blinky service with the `61a885a4-41c3-60d0-9a53-6d652a70d29c` UUID.

> Note that if you want to implement a different or custom Bluetooth® Low Energy service or characteristic, the UUID arrays have to start with the least significant bit (LSB) from left to right.

After uploading the sketch to the Nano Matter, it is time to communicate with it via Bluetooth® Low Energy. To do this Silicon Labs has developed a **mobile app** that you can download from here:

- Android
- iOS

Open the **Simplicity Connect** app on your smartphone, in the lower menu, navigate to **Demo** and select **Blinky**:

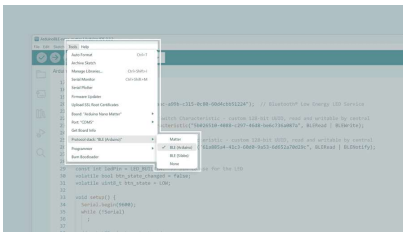


> You can also manage the LED and button status manually from the Scan tab in the lower menu of the app.

Arduino BLE Library

Now let's do the same but using the `ArduinoBLE` library.

In the upper menu, navigate to **Tools > Protocol stack** and select **BLE(Arduino)**.



Enable "BLE(Arduino)" Protocol Stack

For this Bluetooth® Low Energy application example, we are going to control the Nano Matter built-in LED and read the onboard button status, everything from the **Simplicity Connect** app.

```
#include <ArduinoBLE.h>

BLEService ledService("de8a5aac

// Bluetooth® Low Energy LED Sw
BLEByteCharacteristic ledCharac

// Bluetooth® Low Energy Push B
BLEByteCharacteristic switchCha

const int ledPin = LED_BUILTIN;
volatile bool btn_state_changed
volatile uint8_t btn_state = LO

void setup() {
  Serial.begin(9600);
  while (!Serial)
    ;

  // set LED pin to output mode
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, HIGH);

  pinMode(BTN_BUILTIN, INPUT_PU
  attachInterrupt(BTN_BUILTIN,

  // begin initialization
  if (!BLE.begin()) {
```

As you can see, using the `ArduinoBLE` library makes the Bluetooth example more similar to the Bluetooth implementation of other Arduino boards, making it easier to migrate code from one board to another. We end up with a simple `setup()` and `loop()` sketch.

In the `setup()` function the board

alongside the BLE service and characteristics.

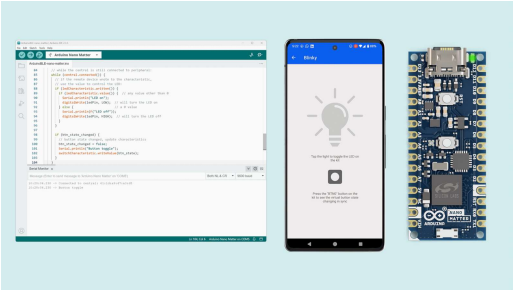
In the `loop()` function we continuously ask if the peripheral is properly connected to a central and then start notifying the push button status and retrieving the app LED status.

After uploading the sketch to the Nano Matter, it is time to communicate with it via Bluetooth® Low Energy. To do this Silicon Labs has developed a **mobile app** that you can download from here:

Android

iOS

Open the **Simplicity Connect** app on your smartphone, in the lower menu, navigate to **Demo** and select **Blinky**:

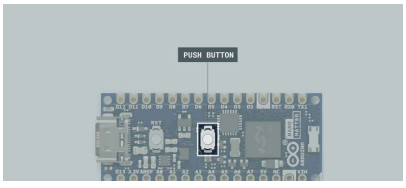


You can also manage the LED and button status manually from the Scan tab in the lower menu of the app.

Onboard User Interface

User Button

The Nano Matter includes an onboard **push button** that can be used as an input by the user. The button is connected to the GPIO `PA0` and can be read using the `BTN_BUILTIN` macro.



Nano Matter Built-in Push Button

The button pulls the input to the ground when pressed, so is important to define the **pull-up resistor** to avoid undesired behavior by leaving the input floating.

Here you can find a complete example code to blink the built-in RGB LED of the Nano Matter:

```
volatile bool button_pressed = f

void handle_button_press();

void setup() {
  Serial.begin(115200);

  pinMode(BTN_BUILTIN, INPUT_PUL
  attachInterrupt(BTN_BUILTIN, &
}

void loop() {
  // If the physical button stat
  if (button_pressed) {
    button_pressed = false;
    Serial.println("Button Press
  }
}

void handle_button_press()
{
  static uint32_t btn_last_press
  if (millis() < btn_last_press
    return;
  }
  btn_last_press = millis();
  button_pressed = true;
}
```

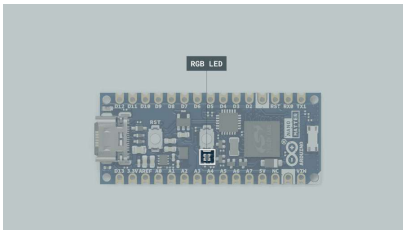
After pressing the push button, you will see a **"Button Pressed!"** message in the Serial Monitor.

RGB LED

The Nano Matter features a built-in RGB LED that can be a visual feedback indicator for the user. The LED is connected through the board GPIO's; therefore, usual digital pins built-in functions can be used to operate the LED colors.

LED Color Segment	Arduino Name	Microcontroller Pin
Red	LEDR or LED_BUILTIN	PC01
Green	LEDG or LED_BUILTIN_1	PC02
Blue	LEDB or LED_BUILTIN_2	PC03

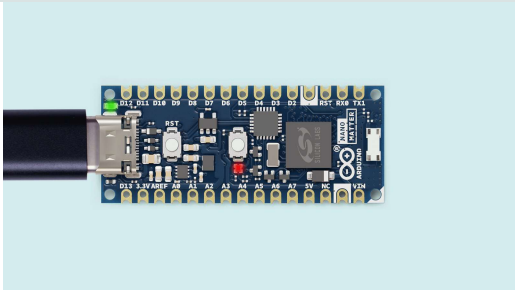
The RGB LED colors are activated with zeros, this means that you need to set to LOW the color segment you want to turn on.



Built-in RGB LED

Here you can find a complete example code to blink the built-in RGB LED of the Nano Matter:

```
void setup() {
  // initialize LED digital pins
  pinMode(LEDR, OUTPUT);
  pinMode(LEDG, OUTPUT);
  pinMode(LEDB, OUTPUT);
}

// the loop function runs over an
void loop() {
  digitalWrite(LEDR, LOW);  // tu
  digitalWrite(LEDG, HIGH);  // t
  digitalWrite(LEDB, HIGH);  // t
  delay(1000);
  digitalWrite(LEDR, HIGH);   //
  digitalWrite(LEDG, LOW);   // t
  digitalWrite(LEDB, HIGH);   //
  delay(1000);
  digitalWrite(LEDR, HIGH);   //
  digitalWrite(LEDG, HIGH);   //
  digitalWrite(LEDB, LOW);   // t
  delay(1000);
}
```



Pins

Digital Pins

The Nano Matter has **22 digital pins**, mapped as follows:

Microcontroller Pin	Arduino Pin Mapping	Pin Functionality
PB00	A0 / DAC0	GPIO / ADC / DAC
PB02	A1 / DAC2	GPIO / ADC / DAC
PB05	A2	GPIO / ADC
PC00	A3	GPIO / ADC
PA06	A4 / SDA	I2C / GPIO / ADC
PA07	A5 / SCL	I2C / GPIO / ADC
PB01	A6 / DAC1	GPIO / ADC / DAC
PB03	A7 / DAC3	GPIO / ADC / DAC
PB04	D13 / SCK	SPI / GPIO / ADC
PA08	D12 / MISO	SPI / GPIO / ADC
PA09	D11 / MOSI	SPI / GPIO / ADC
PD05	D10 / SS	SPI / GPIO
PD04	D9	GPIO
PD03	D8	GPIO / ADC
PD02	D7	GPIO / ADC

Microcontroller Pin	Arduino Pin Mapping	Pin Functionality
PC08	D5 / SCL1	I2C / GPIO / ADC
PC07	D4 / SDA1	I2C / GPIO / ADC
PC06	D3 / SS1	GPIO / SPI / ADC
PA03	D2 / SCK1	SPI / GPIO
PA05	D1 / PIN_SERIAL_RX1 / MISO1	UART / SPI / GPIO / ADC
PA04	D0 / PIN_SERIAL_TX1 / MOSI1	UART / SPI / GPIO / ADC

The digital pins of the Nano Matter can be used as inputs or outputs through the built-in functions of the Arduino programming language.

The configuration of a digital pin is done in the `setup()` function with the built-in function `pinMode()` as shown below:

```
// Pin configured as an input
pinMode(pin, INPUT);

// Pin configured as an output
pinMode(pin, OUTPUT);

// Pin configured as an input, int
pinMode(pin, INPUT_PULLUP);
```

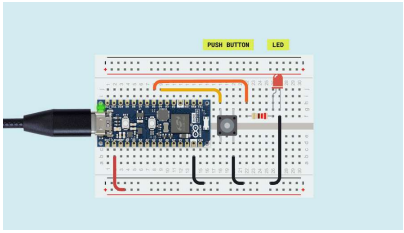
The state of a digital pin, configured as an input, can be read using the built-in function `digitalRead()` as shown below:

```
// Read pin state, store value in
state = digitalRead(pin);
```

The state of a digital pin, configured as an

```
// Set pin on
digitalWrite(pin, HIGH);

// Set pin off
digitalWrite(pin, LOW);
```

The example code shown below uses digital pin `D5` to control an LED and reads the state of a button connected to digital pin `D4`:



Digital I/O example wiring

```
// Define button and LED pin
int buttonPin = D4;
int ledPin = D5;

// Variable to store the button
int buttonState = 0;

void setup() {
  // Configure button and LED pi
  pinMode(buttonPin, INPUT_PULLU
  pinMode(ledPin, OUTPUT);

  // Initialize Serial communica
  Serial.begin(115200);
}

void loop() {
  // Read the state of the butto
  buttonState = digitalRead(butt

  // If the button is pressed, t
  if (buttonState == LOW) {
    digitalWrite(ledPin, HIGH);
    Serial.println("- Button is
  } else {
    // If the button is not pres
    digitalWrite(ledPin, LOW);
    Serial.println("- Button is
```

All the digital and analog pins of the Nano Matter can be used as PWM (Pulse Width Modulation) pins.

> You can only use 5 PWM outputs simultaneously.

This functionality can be used with the built-in function `analogWrite()` as shown below:

```
analogWrite(pin, value);
```

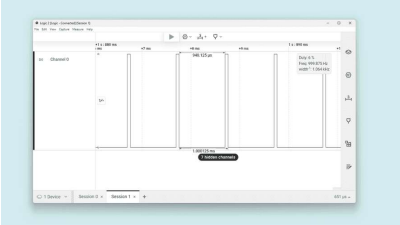
By default, the output resolution is 8 bits, so the output value should be between 0 and 255. To set a greater resolution, do it using the built-in function `analogWriteResolution` as shown below:

```
analogWriteResolution(bits);
```

Using this function has some limitations, for example, the PWM signal frequency is fixed at 1 kHz, and this could not be ideal for every application.

Here is an example of how to create a **1 kHz** variable duty-cycle PWM signal:

```
const int analogInPin = A0;    //
const int pwmOutPin = D13;   // F

int sensorValue = 0;  // value r
int outputValue = 0;  // value c

void setup() {
  // initialize serial communica
  Serial.begin(115200);
  analogWriteResolution(12);
}

void loop() {
  // read the analog in value:
  sensorValue = analogRead(analo
  // map it to the range of the
  outputValue = sensorValue;
  // change the analog out value
  analogWrite(pwmOutPin, outputV

  // print the results to the Se
  Serial.print("sensor = ");
  Serial.print(sensorValue);
  Serial.print("\t output = ");
  Serial.println(outputValue);

  // wait 2 milliseconds before
  // converter to settle after t
```



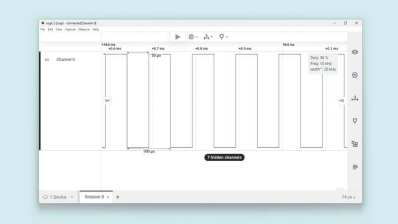
PWM output signal using the PWM at a lower level

If you need to work with a **higher frequency** PWM signal, you can do it with the following PWM class-specific function:

```
PWM.frequency_mode(output_pin, fre
```

Here is an example of how to create a **10 kHz** fixed duty-cycle PWM signal:

```
const int analogOutPin = D13;  //

void setup() {
  analogWriteResolution(12);
}

void loop() {
  PWM.frequency_mode(analogOutPin,
}
```



PWM output signal using the PWM at a lower level

Analog Input Pins (ADC)

The Nano Matter has **20 analog input pins**, mapped as follows:

Microcontroller Pin	Arduino Pin Mapping	Pin Functionality
PB00	A0 / DAC0	GPIO / ADC / DAC
PB02	A1 / DAC2	GPIO / ADC / DAC
PB05	A2	GPIO / ADC
PC00	A3	GPIO / ADC
PA06	A4 / SDA	I2C / GPIO / ADC
PA07	A5 / SCL	I2C / GPIO / ADC
PB01	A6 / DAC1	GPIO / ADC / DAC
PB03	A7 / DAC3	GPIO / ADC / DAC
PB04	D13 / SCK	SPI / GPIO / ADC
PA08	D12 / MISO	SPI / GPIO /

Microcontroller Pin	Arduino Pin Mapping	Pin Functionality
PA09	D11 / MOSI	SPI / GPIO / ADC
PD03	D8	GPIO / ADC
PD02	D7	GPIO / ADC
PC09	D6	GPIO / ADC
PC08	D5 / SCL1	I2C / GPIO / ADC
PC07	D4 / SDA1	I2C / GPIO / ADC
PC06	D3 / SS1	GPIO / SPI / ADC
PA03	D2 / SCK1	SPI / GPIO
PA05	D1 / PIN_SERIAL_RX1 / MISO1	UART / SPI / GPIO / ADC
PA04	D0 / PIN_SERIAL_TX1 / MOSI1	UART / SPI / GPIO / ADC

Digital I/O's can also be used as analog inputs except for `D9` and `D10`.

Analog input pins can be used through the built-in functions of the Arduino programming language.

Nano Matter ADC resolution is fixed to **12 bits** and cannot be changed by the user.

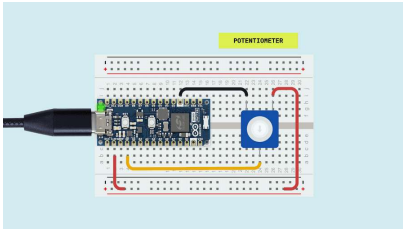
The Nano Matter ADC reference voltage is 3.3 V by default, it can be configured using the function `analogReference()` with the following arguments:

Argument	Description
AR_INTERNAL1V2	Internal 1.2V reference
AR_EXTERNAL_1V25	External 1.25V reference
AR_VDD	VDD (unbuffered to ground)
AR_08VDD	0.8 * VDD (buffered to ground)

To set a different analog reference from the

```
analogReference(AR_INTERNAL1V2);
```

The example code shown below reads the analog input value from a potentiometer connected to `A0` and displays it on the IDE Serial Monitor. To understand how to properly connect a potentiometer to the Nano Matter, take the following image as a reference:



ADC input example wiring

```
int sensorPin = A0;    // select t

int sensorValue = 0;  // variable

void setup() {
  Serial.begin(115200);
}

void loop() {
  // read the value from the sens
  sensorValue = analogRead(sensor

  Serial.println(sensorValue);
  delay(100);
}
```

Analog Output Pins (DAC)

The Nano Matter has **two DACs** with two channels each, mapped as follows:

Microcontroller Pin	Arduino Name	Board Pin Output	Peripheral
PB00	DAC0	A0	DAC0_CH0
PB01	DAC1	A6	DAC0_CH1
PB02	DAC2	A1	DAC1_CH0

The digital-to-analog converters of the Nano Matter can be used as outputs through the built-in functions of the Arduino programming language.

The DAC output resolution can be configured from 8 to 12 bits using the `analogWriteResolution()` function as follows:

```
analogWriteResolution(12);  // ent
```

The DAC voltage reference can be configured using the `analogReferenceDAC()` function. The available setups are listed below:

Argument	Description
DAC_VREF_1V25	Internal 1.25V reference
DAC_VREF_2V5	Internal 2.5V reference
DAC_VREF_AVDD	Analog VDD
DAC_VREF_EXTERNAL_PIN	External AREF pin

```
analogReferenceDAC(DAC_VREF_2V5);
```

To output an analog voltage value through a DAC pin, use the `analogWrite()` function with the **DAC channel** as an argument. See the example below:

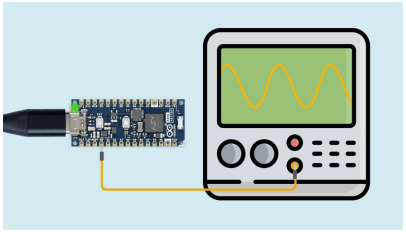
```
analogWrite(DAC0, value);   // the
```

If a normal GPIO is passed to the analogWrite() function, the output will be a PWM signal.

The following sketch will create a **sine** wave signal in the `A0` Nano Matter pin:

```
float sample_rate = 9600.0, freq
int npts = sample_rate / freq;

void setup()
{
  Serial.begin(115200);
  // Set the DAC resolution to 12
  analogWriteResolution(12);
  // Select the 1.25V reference v
  analogReferenceDAC(DAC_VREF_1V2
}

void loop()
{

  for (int i = 0; i < npts; i++)
    int x = 2000 + 1000.0 * sin(2
    analogWrite(DAC0, x);
    delayMicroseconds(1.0E6 / sam
  }
}
```

The DAC output should look like the image below:

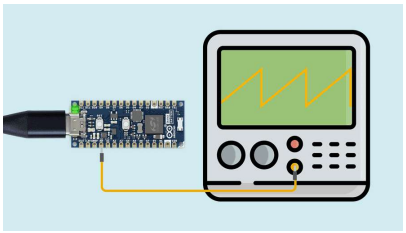


DAC sine wave output

The following sketch will create a **sawtooth** wave signal in the `A0` Nano Matter pin:

```
void setup()
{
  Serial.begin(115200);
  // Set the DAC resolution to 8
  analogWriteResolution(8);
  // Select the 2.5V reference vo
  analogReferenceDAC(DAC_VREF_2V5
}

void loop()
{
  static int value = 0;
  analogWrite(DAC0, value);
  Serial.println(value);

  value++;
  if (value == 255) {
    value = 0;
  }
}
```

The DAC output should look like the image below:



DAC sawtooth wave output

Communication

This section of the user manual covers the different communication protocols that are supported by the Nano Matter, including the Serial Peripheral Interface (SPI), Inter-Integrated Circuit (I2C) and Universal Asynchronous Receiver-Transmitter (UART). The Nano Matter features dedicated pins for each communication protocol, simplifying the connection and communication with different components, peripherals, and sensors.

SPI

The Nano Matter supports SPI communication,

counts with two SPI interfaces and the pins used in the Nano Matter for the SPI communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping
PD05	SS or D10
PA09	MOSI or D11
PA08	MISO or D12
PB04	SCK or D13
PA04	MOSI1 or D0
PA05	MISO1 or D1
PA03	SCK1 or D2
PC06	SS1 or D3

Please, refer to the **board pinout section** of the user manual to localize them on the board.

You can not use **SPI1** and **UART** interfaces at the same time because they share pins.

Include the `SPI` library at the top of your sketch to use the SPI communication protocol. The SPI library provides functions for SPI communication:

```
#include <SPI.h>
```

In the `setup()` function, initialize the SPI peripheral, define and configure the chip select (`SS`) pin:

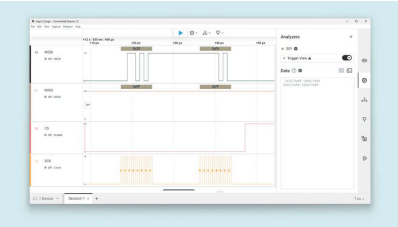
Use SPI.begin() for SPI0 and SPI1.begin() for SPI1.

```
void setup() {
  // Set the chip select pin as o
  pinMode(SS, OUTPUT);

  // Pull the CS pin HIGH to unse
  digitalWrite(SS, HIGH);

  // Initialize the SPI communica
  SPI.begin();
```

To transmit data to an SPI-compatible device, you can use the following commands:

```
// Replace with the target device
byte address = 0x35;

// Replace with the value to send
byte value = 0xFA;

// Pull the CS pin LOW to select
digitalWrite(SS, LOW);

// Send the address
SPI.transfer(address);

// Send the value
SPI.transfer(value);

// Pull the CS pin HIGH to unsele
digitalWrite(SS, HIGH);
```

The example code above should output this:



SPI logic data output

I2C

The Nano Matter supports I2C communication, which enables data transmission between the board and other I2C-compatible devices. The Nano Matter features two I2C interfaces and the pins used in the Nano Matter for the I2C communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping
PA06	SDA or A4
PA07	SCL or A5
PC07	SDA1 or D4
PC08	SCL1 or D5

Please, refer to the board pinout section of the user manual to localize them on the board.

To use I2C communication, include the `Wire` library at the top of your sketch. The `Wire` library provides functions for I2C communication:

```
#include <Wire.h>
```

In the `setup()` function, initialize the I2C library:

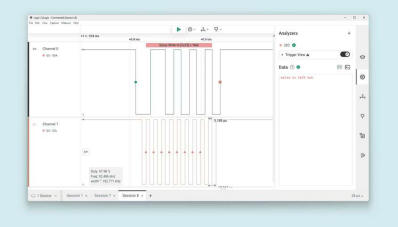
> Use Wire.begin() for I2C0 and Wire1.begin() for I2C1.

```
// Initialize the I2C communicatio
Wire.begin();
```

To transmit data to an I2C-compatible device, you can use the following commands:

```
// Replace with the target device
byte deviceAddress = 0x35;

// Replace with the appropriate i
byte instruction = 0x00;

// Replace with the value to send
byte value = 0xFA;

// Begin transmission to the targ
Wire.beginTransmission(deviceAddr

// Send the instruction byte
Wire.write(instruction);

// Send the value
Wire.write(value);

// End transmission
Wire.endTransmission();
```

The output data should look like the image



I2C output data

To read data from an I2C-compatible device, you can use the `requestFrom()` function to request data from the device and the `read()` function to read the received bytes:

```
// The target device's I2C addres
byte deviceAddress = 0x1;

// The number of bytes to read
int numBytes = 2;

// Request data from the target d
Wire.requestFrom(deviceAddress, n

// Read while there is data avail
while (Wire.available()) {
  byte data = Wire.read();
}
```

UART

The Nano Matter features two UARTs. The pins used in the Nano Matter for the UART communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping
PA05	PIN_SERIAL_RX1 or D1
PA04	PIN_SERIAL_TX1 or D0
PC05	PIN_SERIAL_RX (internal)
PC04	PIN_SERIAL_TX (internal)

Please, refer to the **board pinout section** of the user manual to localize them on the board.

You can not use **UART** and **SPI1** interfaces at the same time because they share pins.

Use Serial.begin() for UART0 (internal) and Serial1.begin() for UART1 (exposed).

To begin with UART communication, you will need to configure it first. In the `setup()` function, set the baud rate (bits per second) for UART communication:

```
// Start UART communication at 115
Serial1.begin(115200);
```

To read incoming data, you can use a `while()` loop to continuously check for available data and read individual characters. The code shown below stores the incoming characters in a String variable and processes the data when a line-ending character is received:

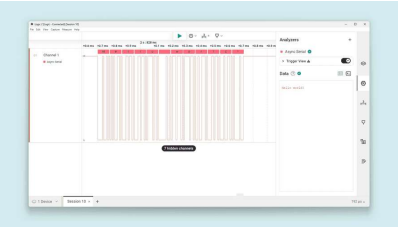
```
// Variable for storing incoming
String incoming = "";

void loop() {
  // Check for available data and
  while (Serial1.available()) {
    // Allow data buffering and r
    delay(2);
    char c = Serial1.read();

    // Check if the character is
    if (c == '\n') {
      // Process the received dat
      processData(incoming);

      // Clear the incoming data
      incoming = "";
    } else {
      // Add the character to the
      incoming += c;
    }
  }
}
```

To transmit data to another device via UART, you can use the `write()` function:

```
// Transmit the string "Hello worl
Serial1.write("Hello world!");
```

You can also use the `print` and `println()` to send a string without a newline character or followed by a newline character:

```
// Transmit the string "Hello worl
Serial1.print("Hello world!");

// Transmit the string "Hello worl
Serial1.println("Hello world!");
```



Serial communication example

Support

If you encounter any issues or have questions while working with the Nano Matter, we provide various support resources to help you find answers and solutions.

Help Center

Explore our **Help Center**, which offers a comprehensive collection of articles and guides for the Nano Matter. The Arduino Help Center is designed to provide in-depth technical assistance and help you make the most of your device.

Nano Matter Help Center page

Forum

Join our community forum to connect with

an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Nano Matter.

Nano Matter category in the Arduino Forum

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We're happy to help you with any issues or inquiries about the Nano Matter.

Contact us page

Suggest changes

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.

Need support?

Help Center
Ask the Arduino Forum
Discover Arduino Discord

License

Was this article helpful?